1 docker 命令
- 1.1 镜像命令
- 1.2 容器命令
- 1.3 查看日志
- 1.4 docker推送

2 shell 脚本
- 2.1 后台启动脚本
- 2.2 强制终止进程（慎用）

3 linux 命令
- 3.1 基础命令
  - 3.1.1 目录相关
  - 3.1.2 权限相关
  - 3.1.3 systemctl
  - 3.1.4 ln
  - 3.1.5 日期
  - 3.1.6 网络下载与传输
  - 3.1.7 端口
  - 3.1.8 进程
  - 3.1.9 主机名
- 3.2 主机资源监控
- 3.3 快捷键
- 3.4 其他补充
  - 3.4.1 软件安装
  - 3.4.2 查看端口占用
  - 3.4.3 vim 命令
  - 3.4.4 ulimit
  - 3.4.5 nvidia-smi
  - 3.4.6 du+sort
  - 3.4.7 环境变量
  - 3.4.8 压缩与解压

4 git
- 4.1 上传
- 4.2 下载
- 4.3 分支管理
- 4.4 历史记录
- 4.5 撤销操作
- 4.6 远程操作
- 4.7 标签管理

5 anaconda常用命令
- 5.1 环境管理
- 5.2 包管理
- 5.3 频道管理
- 5.4 其他

6 GPU机器命令备注
- 6.1 移动 conda 环境

6.2 Huggingface相关
6.3 Modelscope相关
6.4 ollama 相关
6.4.1 命令部署 ollama
6.4.2 安装包部署
6.4.3 注册模型（GGUF格式）
6.4.4 注册模型（safetensors）
6.5 vllm安装+启动
6.6 sglang安装+启动
6.7 npu-smi info
7 mcp
7.1 uv
7.2 MCP极简客户端搭建流程

1 docker 命令

1.1 镜像命令

- 镜像相关命令
    1. 查看本地镜像

```
docker images
```

    2. 搜索镜像仓库

```
docker search <名称关键字>
```

    3. 下载镜像

```
docker pull <镜像名:tag版本>
```

    4. 上传镜像

```
docker push <镜像名:tag版本>
```

    5. 删除镜像

```
docker rmi <镜像名:tag版本>
```

    6. **打包本地镜像文件(单个+多个)**

```
docker save -o <输出文件路径> <镜像名:tag版本>
docker save -o images.tar image_name1:tag1 image_name2:tag2 ...
```

    7. 导入本地镜像文件

```
docker load -i <加载文件路径>
docker load -i images.tar
```

1.2 容器命令

- 容器相关的命令
    1. 查看本地容器

```
docker ps
```

    2. 查看所有容器（包括未运行）

```
docker ps -a
```

3. 启动容器

```
docker start <容器ID>
```

4. 停止容器

```
docker stop <容器ID>
```

5. 删除容器

```
docker rm <容器ID>
```

6. 进入容器内部

```
docker exec -it <容器ID> /bin/bash
docker exec -it <容器ID> /bin/sh
```

7. **容器创建成镜像**

```
docker commit -a "作者信息" -m "log信息" <容器ID> <目标镜像名称: tag版本>
```

8. 拷贝文件到容器

```
docker cp <文件目录> <容器ID>:<目标目录>
```

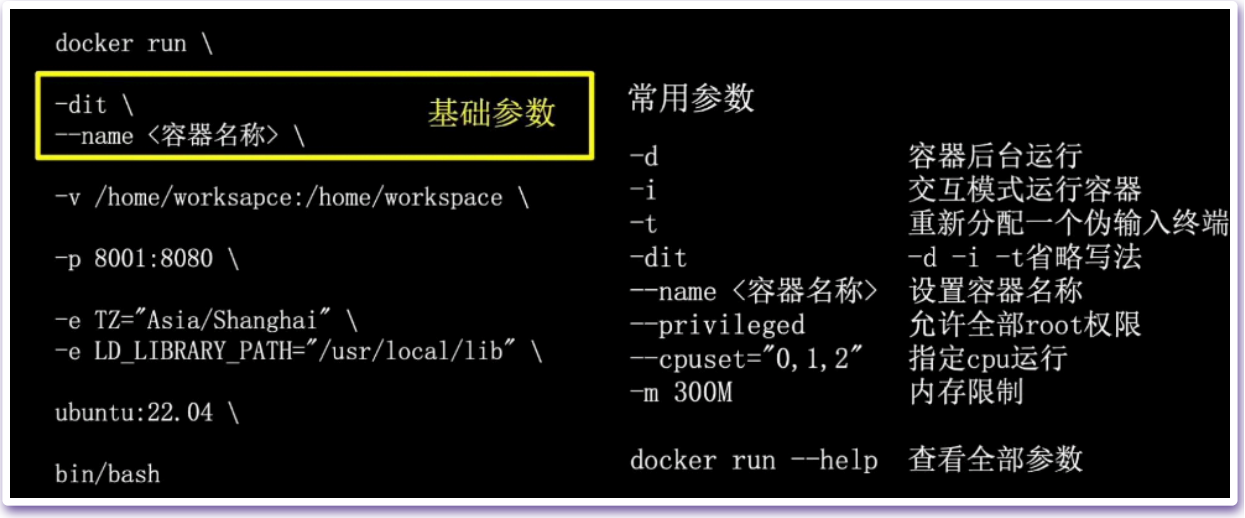
9. 拷贝容器文件到宿主机

```
docker cp <容器ID>:<文件目录> <宿主机目标目录>
```

10. 更新容器设置

```
docker update <容器ID> <相关设置>
```

- 创建容器命令- 基础参数



- 创建容器命令- 挂载宿主机目录

```
docker run \

-d \
--name <容器名称> \

-v /home/worksapce:/home/workspace \

-p 8001:8080 \

-e TZ="Asia/Shanghai" \
-e LD_LIBRARY_PATH="/usr/local/lib" \

ubuntu:22.04 \

bin/bash
```

挂载宿主机目录

-v /home/worksapce:/home/workspace

宿主机目录　　容器内目录

容器可直接操作宿主机此目录下的文件

- 创建容器命令- 网络设置

```
docker run \

-dit \
--name <容器名称> \

-v /home/worksapce:/home/workspace \

-p 8001:8080 \

-e TZ="Asia/Shanghai" \
-e LD_LIBRARY_PATH="/usr/local/lib" \

ubuntu:22.04 \

bin/bash
```

网络设置

设置说明

-p <宿主机端口:容器端口>	端口映射
-h <容器hostname>	指定hostname
--ip 192.168.0.10	指定容器IP
--network=host	共用宿主机网络 其他网络设置将不生效

- 创建容器命令- 环境变量

```
docker run \

-dit \
--name <容器名称> \

-v /home/worksapce:/home/workspace \

-p 8001:8080 \

-e TZ="Asia/Shanghai" \
-e LD_LIBRARY_PATH="/usr/local/lib" \

ubuntu:22.04 \

bin/bash
```

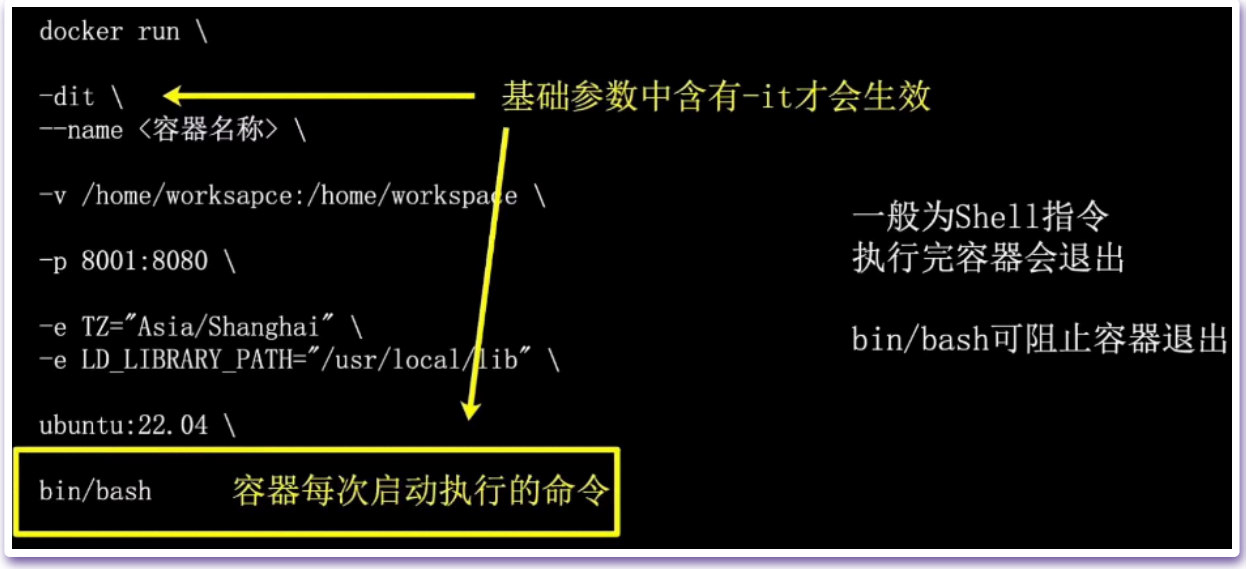
环境变量

设置说明

-e TZ="Asia/Shanghai"　　设置时区

常规Linux设置环境变量方法是不生效的

- 创建容器命令- 容器每次启动执行的命令



1.3 查看日志

- **查看容器的所有日志**（常用）

```
docker logs <container_id_or_name>
```

- **实时查看容器的日志**(类似 tail -f ,常用)

```
docker logs -f <container_id_or_name>
```

- 查看容器的最后几行日志

```
docker logs --tail <number> <container_id_or_name>
```

- 查看从某个时间点开始的日志

```
docker logs --since <timestamp> <container_id_or_name>
```

- 查看到某个时间点为止的日志

```
docker logs --until <timestamp> <container_id_or_name>
```

- 查看日志并显示时间戳

```
docker logs -t <container_id_or_name>
```

1.4 docker推送

- 推送新生成的 docker 镜像到 dockerhub 的步骤

```
# 1. 登录到 Docker Hub
docker login
```

```

# 2. 运行镜像
docker run -d --name my_container my_image:1.0

# 3. 为镜像打标签
docker tag image_id:1.0 my_repo/my_image:1.0

# 4. 提交容器（从容器创建新镜像--可选）
# 假设你在容器中进行了一些修改，并希望保存为新的镜像
docker commit container_id my_repo/my_image:1.1

# 5. 推送镜像到远程仓库
docker push my_repo/my_image:1.1
```

2 shell 脚本

2.1 后台启动脚本

- 用于后台启动 python 脚本用

```
# 找到所有匹配的进程ID并逐一杀死
for pid in $(lsof -i:8069 -t); do
    echo "Killing process $pid"
    kill -9 "$pid"
done


nohup bash -c "source /home/ubuntu/anaconda3/bin/activate rag-search && python3 rag_main.py" > logs/output.log 2>&1 &

sleep 5s

pids=$(ps -ef | grep 'rag_main' | grep -v grep | awk '{printf("%s ", $2)}')

for pid in ${pids}; do
    echo "进程启动成功 $pid"
done
```

需修改地方

1. 端口号
2. anaconda地址、anaconda 虚拟环境名称
3. python 文件名称

2.2 强制终止进程（慎用）

- 以强制终止 python 进程为例子

```
ps -ef | grep python | grep -v grep | awk '{print $2}' | xargs kill -9
```

命令解释

1. `ps -ef`：`ps` 命令用于显示当前系统运行的进程，包括其他用户启动的进程；
    - `-e` 选项表示选择所有进程。
    - `-f` 选项表示使用全格式显示，包括更多的进程信息。
2. `grep python`：
    - `grep` 命令用于在文本中搜索指定的模式。

        - 这里搜索包含 "python" 的行，即查找所有与 Python 相关的进程。
    3. `grep -v grep`：
        - `-v` 选项表示反向选择，即排除包含指定模式的行。
        - 这里排除包含 "grep" 的行，以避免终止正在执行 `grep` 命令的进程。
    4. `awk '{print $2}'`：
        - `awk` 是一种文本处理工具，用于对文本进行格式化和提取。
        - `{print $2}` 表示打印每行的第二个字段。
        - 在 `ps -ef` 的输出中，第二个字段是进程的 PID（进程 ID）。
    5. `xargs kill -9`：
        - `xargs` 命令用于从标准输入构建和执行命令行。
        - `kill -9` 命令用于强制终止进程。
        - `-9` 选项表示发送 SIGKILL 信号，强制终止进程。
- 综合起来，这个命令的执行步骤如下：
    1. 列出所有进程。
    2. 过滤出包含 "python" 的进程。
    3. 排除包含 "grep" 的行。
    4. 提取每个 Python 进程的 PID。
    5. 使用 `kill -9` 强制终止这些进程。

---

3 linux 命令

3.1 基础命令

3.1.1 目录相关

- 切换到**家目录**(/home/用户名)

```
cd
cd ~
```

- 切换到**上级目录**

```
cd ../..
```

- **复制文件夹**

```
CP -r 复制文件夹
```

- **递归创建**

```
midir  -p  递归创建
```

- **删除文件夹**

```
rm -r 文件夹
```

`-f` 表示强制（root 用户才会有提示）

more +Linux 路径 查看文件内容，空格单页，q 退出

MV 1.txt 2.txt 可用于改名

exit 可从 root 退回到普通用户

find 起始路径 -name 被查找文件名

find 起始路径 -size + -n kMB(注意大小写)

find /use -size +10k 找出/usr 大于 10k 的文件

gerp -n 关键字 文件路径

WC -l -c -m – w 文件路径

管道符 将左边的结果作为右边的内容输入

echo 反引号中的内容将被当做命令执行 而不是普通字符

重定向符

>覆盖

>>追加

3.1.2 权限相关

- 切换用户（switch user）

```
su - 用户名
```

- 临时赋予 root 授权

```
sudo
```

- 创建用户组

```
groupadd 用户组名
```

- 删除用户组

```
groupdel 用户组名
```

- 创建用户并指定 home 目录和用户组

```
useradd -d 指定 home 目录 -g 指定用户组 用户名
```

- 删除用户并删除 home 目录

```
userdel -r 用户名
```

- 查看用户所属组

```
id 用户名
```

- 将用户添加到用户组

```
usermod -aG 用户组 用户名
```

- 查看系统中所有用户

```
getent passwd
```

- 查看系统中所有用户组

```
getent group
```

- 递归修改文件或文件夹的所有者和用户组

```
chown -R 用户:用户组 文件/文件夹
```

-R 表示对文件夹内全部内容应用相同规则

- 将文件的所属用户修改为 root

```
chown root 111.txt
```

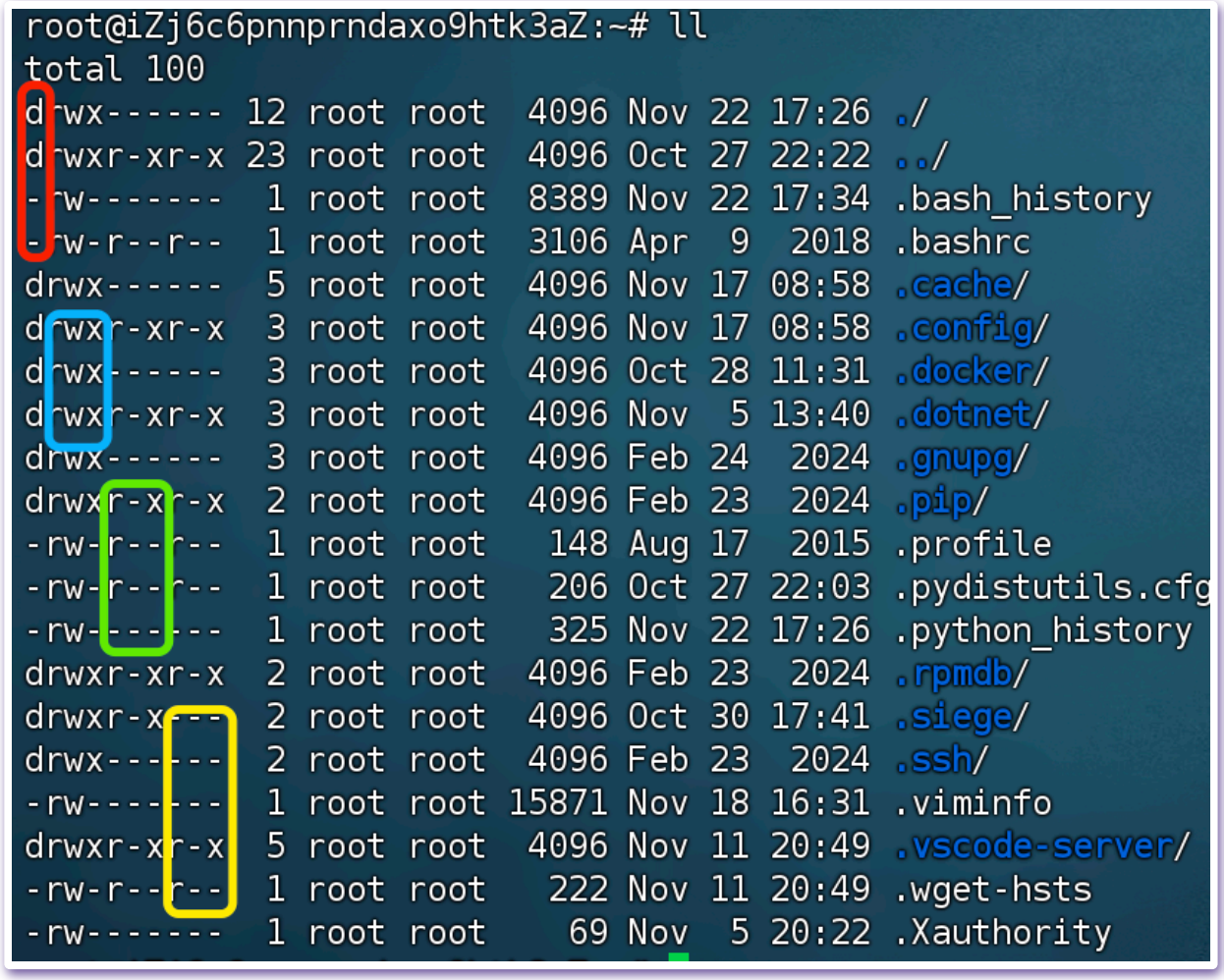
- 只修改文件的所属用户组

```
chown :staff 111.txt
```

- 递归修改文件夹及其所有子文件和子文件夹的所有者和用户组为 root

```
chown -R root:root test
```

- 备注：



类似：d-x--wx---

1. 第一个 d 表示文件夹-表示文件 l 表示全链接
2. 另每三个分别是 用户 用户组 其他用户权限
3. x 表示执行（1），w 表示写（2），r 表示 读（4）

3.1.3 systemctl

- 用于管理系统服务和其它系统资源，命令格式如下

```
systemctl start/restart/stop/status/enable/disable 服务名
```

start 启动服务、restart 重启服务、stop 停止服务、status 查看服务状态（isrunning or dead）、enable 开机自启动、disable 禁用开机自启动

3.1.4 ln

- 创建软连接（类似 windows 的快捷方式）

```
ln -s 参数1 参数2
```

1. -s : 创建软连接
2. 参数 1： 被连接的文件或文件夹
3. 参数 2：要连接去的目的地

3.1.5 日期

- date命令

```
date -d +格式化字符串
```

date命令进行日期加减,-d选项可以和 格式化字符串配合一起使用味

-d选项，可以按照给定的字符串显示日期

1. date -d "+1 day" +%Y%m%d *显示后一天的日期
2. date -d "-1 day" +%Y%m%d * 显示前一天的日期
3. date -d "-1 month" +%Y%m%d * 显示上一月的日期
4. date -d "+1 month" +%Y%m%d * 显示下一月的日期
5. date -d "-1 year" +%Y%m%d * 显示前一年的日期
6. date -d "+1 year" +%Y%m%d * 显示下一年的日期

- 其中支持的时间标记为:
  year年
  month月
  day天
  hour小时
  minute分钟
  second秒

3.1.6 网络下载与传输

- ping 命令

```
ping -c num ip 或主机名
```

- wget命令

```
wget -b url
```

-b为回台下载，这时会输出日志到wget-log文件中

- curl命令

```
curl -O url
```

-O 选项用于下载。

3.1.7 端口

- 什么是端口?

  端口是指计算机和外部交互的出入口，可以分为物理端口和虚拟端

  1. 物理端口 : USB、HDMI、 DP、VGA、RJ45等
  2. 虚拟端口 : 操作系统和外部交互的出入口

  IP只能确定计算机，通过端口才能锁定要交互的程序

- 端口的划分

  1. 公认端口: 1 1023, 用于系统内置或常用知名软件绑定使用
  2. 动态端口： 49152 65535，用于临时使用（多用于出口）

- 查看端口占用

  1. 查看指定IP的对外暴露端口

     ```
     nmap IP地址
     ```

  2. 查看本机指定端口号的占用情况

     ```
     netstat -anp | grep 端口号
     ```

3.1.8 进程

- 查看Linux系统中的进程信息

  一般来说，固定用法就是：ps -ef 列出全部进程的全部信息

```
ps -ef

ps -ef | grep python
```

选项：

  1. -e, 显示出全部的进程
  2. -f, 以完全格式化的形式展示信息（展示全部信息）

```
root@iZj6c6pnnprndaxo9htk3aZ:~# ps -ef
UID        PID  PPID  C STIME TTY          TIME CMD
root         1     0  0 Nov05 ?        00:24:59 /sbin/init noibrs splash
root         2     0  0 Nov05 ?        00:00:00 [kthreadd]
root         4     2  0 Nov05 ?        00:00:00 [kworker/0:0H]
root         6     2  0 Nov05 ?        00:00:00 [mm_percpu_wq]
root         7     2  0 Nov05 ?        00:01:48 [ksoftirqd/0]
root         8     2  0 Nov05 ?        00:07:40 [rcu_sched]
root         9     2  0 Nov05 ?        00:00:00 [rcu_bh]
root        10     2  0 Nov05 ?        00:00:13 [migration/0]
root        11     2  0 Nov05 ?        00:00:02 [watchdog/0]
root        12     2  0 Nov05 ?        00:00:00 [cpuhp/0]
root        13     2  0 Nov05 ?        00:00:00 [cpuhp/1]
root        14     2  0 Nov05 ?        00:00:02 [watchdog/1]
root        15     2  0 Nov05 ?        00:00:13 [migration/1]
root        16     2  0 Nov05 ?        00:01:42 [ksoftirqd/1]
root        18     2  0 Nov05 ?        00:00:00 [kworker/1:0H]
```

从左到右分别是：

1. UID: 进程所属的用户IDPID: 进程的进程号
2. PPID: 进程的父ID（启动此进程的其它进程）
3. C：此进程的CPU占用率（百分比）
4. STIME: 进程的启动时间
5. TTY：启动此进程的终端序号，如显示?，表示非终端启动
6. Time: 进程占用CPU时间
7. CMD: 启动路径或启动命令

3.1.9 主机名

- 查看主机名

```
hostname
```

- 修改主机名

```
hostnamectl set-hostname 新的主机名
```

3.2 主机资源监控

- 系统资源监控(查看 CPU 内存使用情况)

```
top
```

- 磁盘信息监控

```
df -h
```

-h 以人性化的显示

- 查看 CPU 磁盘相关信息

```
iostat  -x num1 num2
```

x 显示更多信息，num 1 刷新间隔，num 2 刷新次数。

- 网络状态监控

```
sar  -n DEV num 1  num2
```

-n 查看网络， DEV 查看网络接口。

3.3 快捷键

- 终端快捷键

1. 强制退出

```
ctrl + c
```

2. 退出登录/退出特定环境的登录

   如 python

```
ctrl + d
```

3. 历史命令

```
history
```

4. 快捷写法，在历史命令中自下向上搜索以 p 开头的命令

```
!p
```

5. 输入内容搜索历史命令，收到后 enter 执行，左右键选中

```
ctrl + r
```

6. 光标移动到头

```
ctrl + a
```

7. 光标移动到尾

```
ctrl + e
```

8. 光标向左向右跳一个单词

```
ctrl + 左/右
```

9. 清屏

```
ctrl + l
```

或者

```
clear
```

10. 从当前光标删除到头

```
ctrl u
```

11. 从当前光标删除到 尾部

```
ctrl k
```

3.4 其他补充

3.4.1 软件安装

- centos 上包管理器

```
yum -y install/search/remove 软件名
```

- ubuntu 上包管理器

```
apt -y install/search/remove 软件名
```

- 备注：
    1. cenos 和 ubuntu上命令一致，但是包管理器不一致
    2. cenos 系统安装包后缀为 .rpm, unbutu系统安装包后缀为.deb
    3. -y 选项为自动确认
    4. install 安装，search搜索包，remove 删除包。

3.4.2 查看端口占用

- 查看进程占用（lsof -i :端口号）

```
lsof -i :8069
```

通过该命令查看占用8069端口的进程，找到对应的pid。
系统将显示监听该端口的进程信息，包括进程名称、进程ID、用户和协议等。如果需要更多信息，可以添加-P和-n参数，如

```
lsof -P -n -i :8069
```

3.4.3 vim 命令

- 删除全部

```
:%d
```

- 替换

  替换 a 成 b

  ```
  :%s/a/b/g
  ```

  1. :%s:表示在整个文件范围内进行替换操作
  2. g 表示全局替换，即替换所有匹配项，而不仅仅是每行的第一个匹配项。

  替换目录 /data/apps 为/data/cols

  ```
  :%s/\/data\/apps/\/data\/cols/g
  ```


- 设置 vim 内部编码

  ```
  :set enc=utf-8
  :set encoding=utf-8
  ```

- 启用粘贴模式：(在 docker 实例中如果不能黏贴)

  ```
  :set paste
  ```

- 设置显示行号

  ```
  :set num
  ```

3.4.4 ulimit

- 查看当前的文件描述符限制

  ```
  ulimit -n
  ```

- 临时设置文件描述符的最大数量为 10240

  ```
  limit -n 10240
  ```

  1. `ulimit`：用于显示或设置用户进程的资源限制。
  2. `-n`：指定文件描述符的最大数量。

3.4.5 nvidia-smi

- 查询详细信息

  ```
  nvidia-smi -q
  ```

- 查询指定GPU详细信息

```
nvidia-smi -q -i 0
```

- 查询指定GPU的特定信息

```
nvidia-smi -q -i 0 -d MEMORY
```

- 帮助命令

```
nvidia-smi -h
```

3.4.6 du+sort

- 查看磁盘占用情况

```
du -sh * | sort -h -rss
```

    1. `du -sh *`： (disk usage)列出当前目录下所有目录磁盘占用情况
    2. `sort -hrs`

       `-h` :按照人类可读的格式来排序

       `-r` : reverse,表示反向排序

       `-s` : 表示稳定排序（stable sorting），在排序过程中，对于相等大小的数据元素，保持它们在原始输入中的相对顺序不变

3.4.7 环境变量

- `什么是环境变量?`

  环境变量是一组信息记录，类型是KeyValue型 (名称=值)，用于操作系统运行的时候记录关键信息

  查看当前系统配置的环境变量信息

```
env
```

  取出环境变量的值

```
$
```

- `什么是PATH?`

  环境变量PATH会记录一组目录，目录之间用:隔开。这里记录的是命令的搜索路径，当执行命令会从记录中记录的目录中挨个搜索要执行的命令并执行。可以通过修改这个项目的值，

  加入自定义的命令搜索路径
  如 export PATH=$PATH: 自定义路径

- `如何修改环境变量`?

    1. 临时生效:

```
export 名称=值
```

2. 永久生效:

   针对用户： `/ bashrc` 文件中配置

   针对全部用户： `/etc/Profile` 文件中配置

3. 配置完成 后立即生效命令

```
source
```

- `自定义环境变量PATH`

  环境变量PATH这个项目里面记录了系统执行命令的搜索路径。
  这些搜索路径我们也可以自行添加到PATH中去。

  测试:

  1. 创建 helloworld, 填入 echo 哈哈哈哈哈
  2. 完成上述操作后，随意切换工作目录，执行helloworld命令尝试一下，会发现无法执行
  3. 修改PATH的值,临时修改PATH:

```
export PATH=$PATH:/root/helloworld
```

  4. source后运行 helloworld

3.4.8 压缩与解压

- `常用压缩格式`

  1. **.tar**, 称之为tarball, 归档文件, 即简单的将文件组装到一个.tar的文件内, 并没有太多文件体积的减少, 仅仅是简单的封装
  2. **.gz**, 也常见为.tar.gz, gzip格式压缩文件，即使用gzip压缩算法将文件压缩到一个文件内，可以极大的减少压缩后的体积

  针对这两种格式，使用tar命令均可以进行压缩和解压缩的操作

- tar

```
# 压缩文件
tar -cxvf

# 解压文件
tar -zxvf
```

  1. -c 创建压缩文件, 用于压缩模式
  2. $-v$, 显示压缩、解压过程, 用于查看进度

3. -x, 解压模式
4. -f，要创建的文件，或要解压的文件，-f选项必须在所有选项中位置处于最后一个
5. -z, gzip模式，不使用-Z就是普通的tarball格式
6. -C，选择解压目的地

- zip

  压缩

```
# -r为压缩文件夹时使用
zip [-r] 参数
```

  解压

```
# -d 指定解压目录
unzip [-d] 参数
```

- 7z

  压缩

```
# 添加文件到 7z 压缩包，可指定密码，不指定则在后续交互中输入
7z a [-p<密码>] <目标 7z 文件名.7z> <源 tar 文件名.tar>

7z a -pPafc?2025 perplexica_images.tar.7z perplexica_images.tar

```

  解压

```
# 解压带密码的 7z 文件，可指定密码，不指定则在后续交互中输入
7z x [-p<密码>] <7z 文件名.7z>

7z x -pPafc?2025 perplexica_images.tar.7z
```

1. `a`： 表示添加文件到压缩包，用于压缩模式。
2. `x`： 表示解压压缩包，用于解压模式。
3. `-p`： 用于指定压缩或解压时的密码。若不直接跟密码，执行命令后会提示输入密码，提高安全性。
4. `<目标 7z 文件名.7z>`： 要创建的 7z 压缩包的名称。
5. `<源 tar 文件名.tar>`： 需要压缩的 tar 文件的名称。
6. `<7z 文件名.7z>`： 要解压的 7z 压缩包的名称。

4 git

4.1 上传

- 查看工作目录和暂存区的状态。显示哪些文件被修改、新增或删除，以及哪些文件还没有被提交

```
git status
```

- 暂存所有修改的文件

```
git add .
```

- 提交更改

```
git commit -m 'this is comment'
```

- 推送仓库

```
git push
```

4.2 下载

- 从远程仓库克隆项目到本地

```
git clone <repository_url>
```

只拉最近一次更新的(只有master分支)

```
git clone <repository_url> --depth 1
```

- 从远程仓库拉取最新更改

```
git pull
```

4.3 分支管理

- 查看分支

```
# 列出所有本地分支
git branch

# 列出所有远程分支
git branch -r

# 列出所有本地分支和远程分支
git branch -a
```

- 创建新分支

```
git branch <branch_name>
```

- 切换到指定分支

```
git checkout <branch_name>
```

- 创建并切换到新分支

```
git checkout -b <branch_name>
```

- 删除本地分支

```
git branch -d <branch_name>
```

- 删除远程分支

```
git push origin --delete <branch_name>
```

- 如果本地没有远程分支的话，将远程分支拉下来需要

  创建本地分支并关联远程分支

  例如：创建本地test分支，并与远程test分支关联

```
git checkout -b test origin/test
```

4.4 历史记录

- 查看提交历史

```
git log
```

- 查看简化的提交历史

```
git log --oneline
```

- 查看某个文件的修改历史

```
git log -p <file_name>
```

4.5 撤销操作

- 撤销工作目录中的所有更改

```
git checkout -- .
```

- 撤销暂存区的更改

```
git reset HEAD <file_name>
```

- 撤销上一次提交

```
git reset --soft HEAD^
```

- 撤销上一次提交并丢弃所有更改

```
git reset --hard HEAD^
```

4.6 远程操作

- 查看远程仓库信息

```
git remote -v
```

- 添加远程仓库

```
git remote add origin <repository_url>
```

- 删除远程仓库

```
git remote remove origin
```

- 重命名远程仓库

```
git remote rename <old_name> <new_name>
```

4.7 标签管理

- 查看所有标签

```
git tag
```

- 创建轻量标签

```
git tag <tag_name>
```

- 创建带注释的标签

```
git tag -a <tag_name> -m 'tag message'
```

- 推送标签到远程仓库

```
git push origin <tag_name>
```

- 删除本地标签

```
git tag -d <tag_name>
```

- 删除远程标签

```
1   git push origin --delete <tag_name>
```

---

5 anaconda常用命令

5.1 环境管理

- 创建新环境

```
conda create --name myenv python=3.8
```

这将创建一个名为myenv的新环境，并指定Python版本为3.8。

- 删除环境

```
conda env remove --name myenv
```

或者

```
conda remove --name your_env_name --allconda
```

- 激活环境

```
conda activate myenv
```

在Windows上可能使用：

```
activate myenv
```

- 退出环境

```
conda deactivate
```

在Windows上可能使用：

```
deactivate
```

- 列出所有环境

```
conda env list
```

或者

```
conda info --envs
```

5.2 包管理

- 安装包

```
conda install numpy
```

这将在当前激活的环境中安装numpy包。

- 更新包

```
conda update numpy
```

- 卸载包

```
conda remove numpy
```

- 列出环境中安装的包

```
conda list
```

5.3 频道管理

- 添加频道

```
conda config --add channels conda-forge
```

- 移除频道

```
conda config --remove channels conda-forge
```

- 查看已添加的频道

```
conda config --show channels
```

5.4 其他

- 清除缓存

```
conda clean --all
```

- 查看conda版本

```
conda --version
```

- 更新conda

```
conda update conda
```

6 GPU机器命令备注

- 以**蓝耘**为例子，默认的目录是在

> cd /root/lanyun-tmp

- 学术加速

  开启学术加速

```
source /etc/network_turbo
```

  取消学术加速

```
unset http_proxy && unset https_proxy
```

  限制以下网站使用

> github.com
> githubusercontent.com
> githubassets.com
> huggingface.co

6.1 移动 conda 环境

- 修改配置，将 conda 虚拟环境安装到数据盘

```
mkdir -p /root/lanyun-tmp/conda/pkgs
conda config --add pkgs_dirs /root/lanyun-tmp/conda/pkgs

mkdir -p /root/lanyun-tmp/conda/envs
conda config --add envs_dirs /root/lanyun-tmp/conda/envs
```

```
conda init
source ~/.bashrc
```

6.2 Huggingface相关

- 假设当前目录为

> /root/lanyun-tmp

- 创建文件夹统一存放 `Huggingface` 下载的模型

```
mkdir Hugging-Face
```

- 配置镜像源

```
vim ~/.bashrc
```

填入以下两个，以修改HuggingFace 的镜像源 、模型保存的默认

> export HF_ENDPOINT=**https://hf-mirror.com**
> export HF_HOME=/root/lanyun-tmp/Hugging-Face

重新加载，查看环境变量是否生效

```
source ~/.bashrc

echo $HF_ENDPOINT
echo $HF_HOME
```

- 安装 HuggingFace 官方下载工具

```
pip install -U huggingface_hub
```

- 执行下载模型的命令，以下载 deepseekr1-1.5b为例

> **https://huggingface.co/deepseek-ai/DeepSeek-R1-Distill-Qwen-1.5B**

复制全名

> deepseek-ai/DeepSeek-R1-Distill-Qwen-1.5B

执行下载命令

```
huggingface-cli download --resume-download deepseek-ai/DeepSeek-R1-Distill-Qwen-1.5B
```

- transformers 代码

```
from transformers import AutoModelForCausalLM, AutoTokenizer

model_name = "Qwen/QwQ-32B"

model = AutoModelForCausalLM.from_pretrained(
    model_name,
    torch_dtype="auto",
    device_map="auto"
)
tokenizer = AutoTokenizer.from_pretrained(model_name)

prompt = "How many r's are in the word \"strawberry\""
messages = [
```

```
    {"role": "user", "content": prompt}
]
text = tokenizer.apply_chat_template(
    messages,
    tokenize=False,
    add_generation_prompt=True
)

model_inputs = tokenizer([text], return_tensors="pt").to(model.device)

generated_ids = model.generate(
    **model_inputs,
    max_new_tokens=32768
)
generated_ids = [
    output_ids[len(input_ids):] for input_ids, output_ids in zip(model_inputs.input_ids, generated_ids)
]

response = tokenizer.batch_decode(generated_ids, skip_special_tokens=True)[0]
print(response)

```

6.3 Modelscope相关

- 假设当前目录为

> /root/lanyun-tmp

- 创建文件夹统一存放 `Modelscope` 下载的模型

```
mkdir Model-Scope
```

- 安装 modelscope

```
pip install modelscope --index https://pypi.mirrors.ustc.edu.cn/simple
```

- 执行下载模型的命令，以下载 qwen2.5-0.5b 为例

> **https://modelscope.cn/models/Qwen/Qwen2.5-0.5B-Instruct/files**

复制全名

> Qwen/Qwen2.5-0.5B-Instruct

```
modelscope download --model Qwen/Qwen2.5-0.5B-Instruct --local_dir /root/lanyun-tmp/Model-Scope/Qwen2.5-0.5B

modelscope download --model Qwen/QwQ-32B --local_dir /root/lanyun-tmp/Model-Scope/QwQ-32B
```

1. `modelscope download`：ModelScope 命令行工具的下载命令，用于从 ModelScope 平台下载指定的模型。
2. `--model Qwen/Qwen2.5-0.5B-Instruct`：

   `--model` 参数指定要下载的模型的唯一标识符（Model ID）。

   `Qwen/Qwen2.5-0.5B-Instruct` 模型的 ID，
3. `--local_dir /root/lanyun-tmp/Model-Scope/Qwen2.5-0.5B`：

   `--local_dir` 参数指定模型下载后存放的本地目录路径。

   `/root/lanyun-tmp/Model-Scope/Qwen2.5-0.5B` 是目标目录路径，表示模型将被下载到这个目录中。

- modelscope 代码

```python
from modelscope import AutoModelForCausalLM, AutoTokenizer
model_name = "./Model-Scope/QwQ-32B/"

model = AutoModelForCausalLM.from_pretrained(
    model_name,
    torch_dtype="auto",
    device_map="auto",
    low_cpu_mem_usage=True
)

tokenizer = AutoTokenizer.from_pretrained(model_name)

prompt = "How many r's are in the word \"strawberry\""
messages = [
    {"role": "user", "content": prompt}
]

text = tokenizer.apply_chat_template(
    messages,
    tokenize=False,
    add_generation_prompt=True
)

model_inputs = tokenizer([text], return_tensors="pt").to(model.device)

generated_ids = model.generate(
    **model_inputs,
```

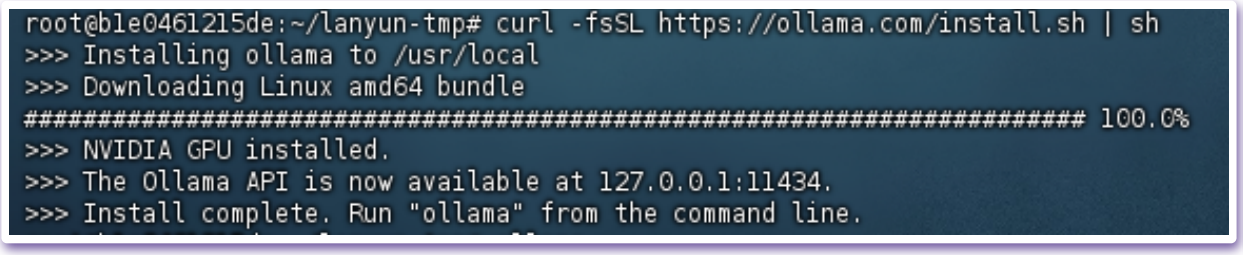
```
    max_new_tokens=32768
)

generated_ids = [
    output_ids[len(input_ids):] for input_ids, output_ids in zip(model_inputs.input_ids, generated_ids)
]

response = tokenizer.batch_decode(generated_ids, skip_special_tokens=True)[0]

print(response)
```

6.4 ollama 相关

6.4.1 命令部署 ollama

- 官网安装命令

```
curl -fsSL https://ollama.com/install.sh | sh
```



- 启动 ollama

```
ollama server
```

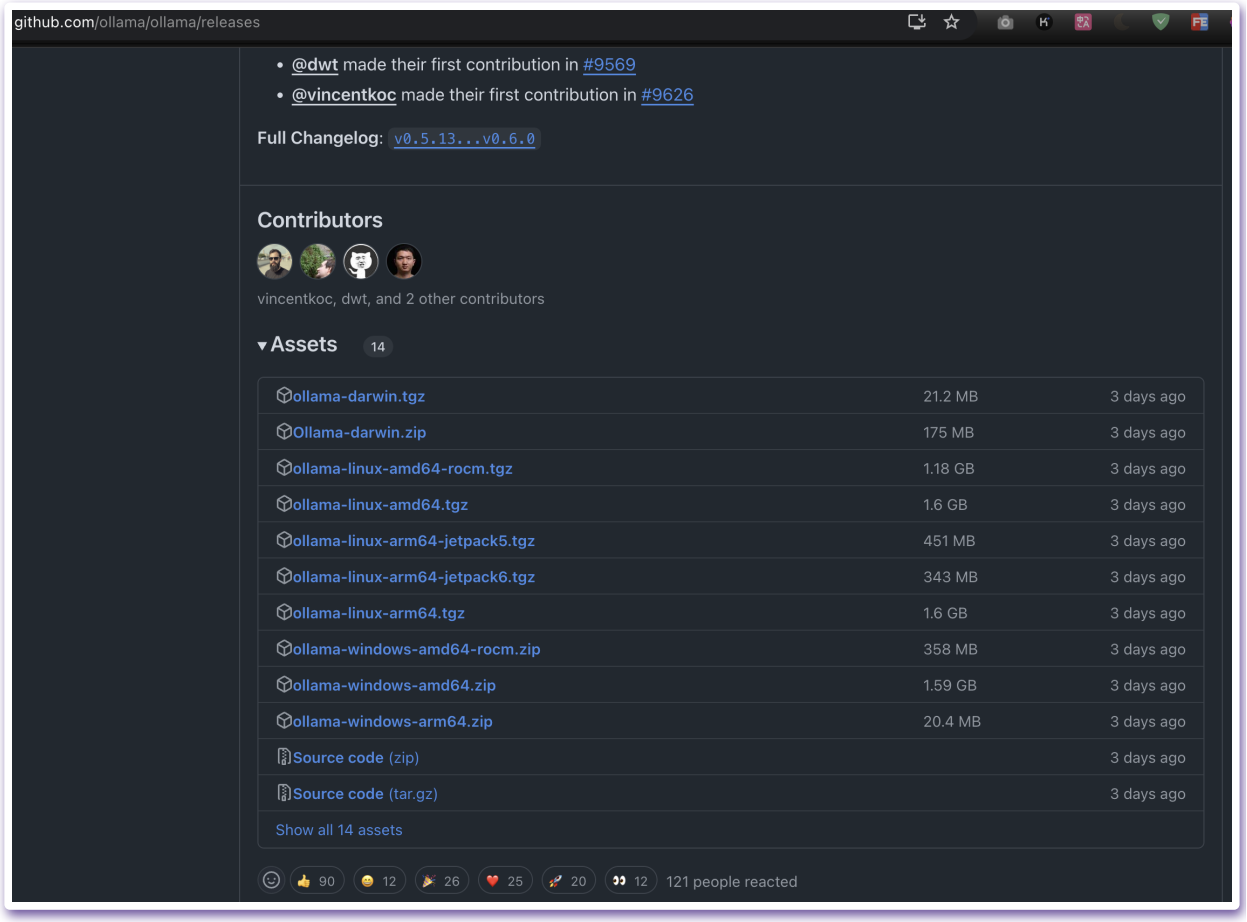
后台启动 ollama 并将 日志写入当前文件夹的 ollama.log 文件夹下

```
nohup ollama serve > ollama.log 2>&1 &
```

6.4.2 安装包部署

- ollama github主页的安装包

> https://github.com/ollama/ollama/releases



点击下载（因网络原因建议点击下载）或者使用 wegt 下载对应 系统版本的 ollama 安装包，下面使用 wget 来演示，我使用的 GPU主机是 ubuntu 的，所以使用 `ollama-linux-arm64.tgz` ,右键复制下载链接，使用 wget 下载

```
wget https://github.com/ollama/ollama/releases/download/v0.6.0/ollama-linux-arm64.tgz
```

```
mkdir ./ollama
tar -zxvf ollama-linux-amd64.tgz -C ./ollama
```

```
chmod +x ollama
```

- 将 bin 目录添加到系统环境变量

```
vim ~/.bashrc
```

添加以下

> export PATH=$PATH:/root/lanyun-tmp/ollama/bin

保存并退出后，输入如下命令来使环境变量生效：

```
source ~/.bashrc
```

- 更换 ollama 模型下载的存放地址

```
vim ~/.bashrc
```

添加如下

> export OLLAMA_MODELS=/root/lanyun-tmp/models

保存并退出后，输入如下命令来使环境变量生效：

```
source ~/.bashrc
```

测试下

```
echo $OLLAMA_MODELS
```

6.4.3 注册模型（GGUF格式）

- cd到模型gguf存放文件夹下，创建文件

```
cd  /root/lanyun-tmp/QwQ-32B-GGUF

vim ModelFile
```

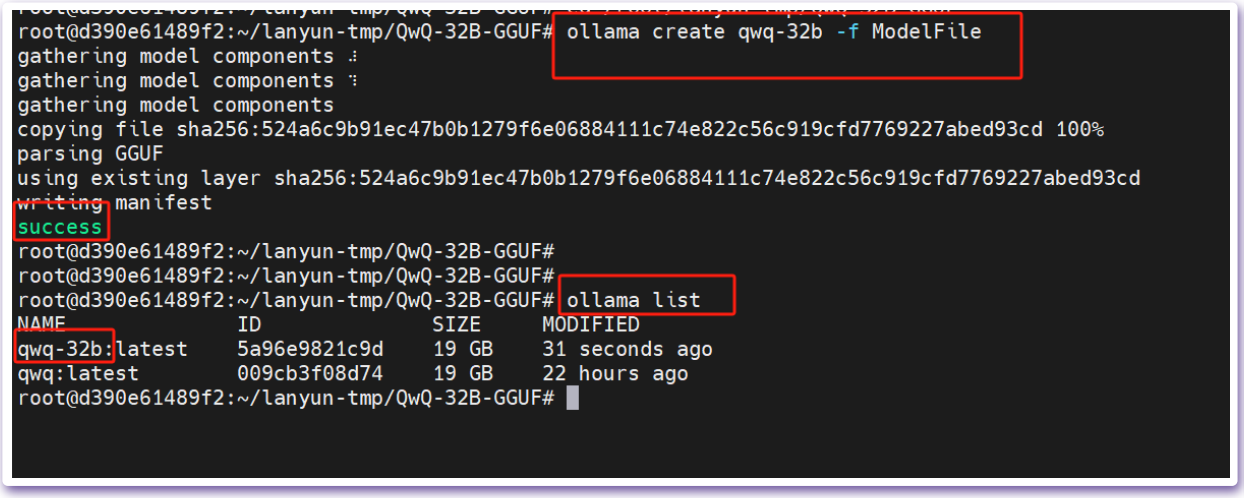
添加如下

> FROM ./qwq-32b-q4_k_m.gguf

- 在当前目录下，使用ollama创建导入模型

```
cd /root/lanyun-tmp/QwQ-32B-GGUF

ollama create qwq-32b -f ModelFile
```

等待成功后即可查看



6.4.4 注册模型（safetensors）

- 由于 Ollama 仅支持 **GGUF** 格式的模型，需要使用 `llama.cpp` 将 `.safetensors` 转换为 `.gguf` 格式：

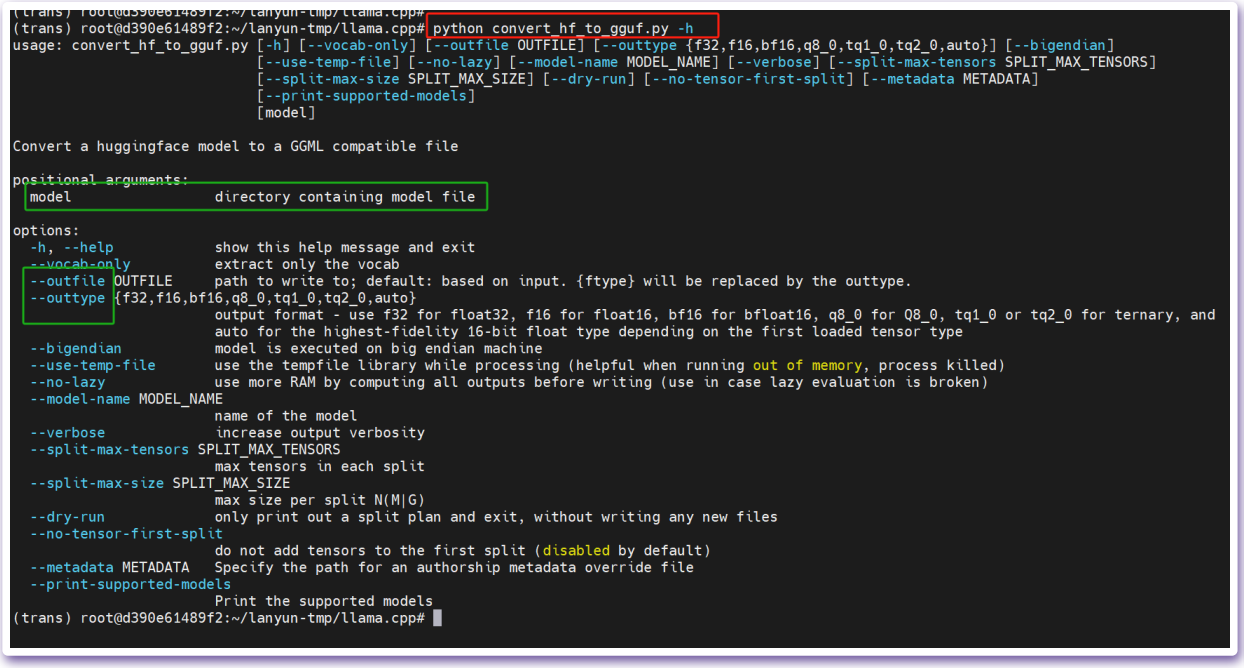
```
cd /root/lanyun-tmp

git clone https://github.com/ggml-org/llama.cpp.git
```

安装依赖

```
cd /root/lanyun-tmp/llama.cpp
pip install -r requirements.txt
```

查看转换safetensors 到gguf格式的py文件输入参数

```
python convert_hf_to_gguf.py -h
```



- 转换

```
export model_path=模型路径
export output_path=转换后路径

echo $model_path
echo $output_path
```

运行下面代码开始转换

```
python convert_hf_to_gguf.py $model_path --outfile $output_path --outtype f32
```

6.5 vllm安装+启动

- 使用conda创建虚拟环境

```
conda create --name vllm python=3.10 -y

conda activate vllm
```

- 下载vllm（指定源，否则极慢）

```
pip install vllm -i https://pypi.tuna.tsinghua.edu.cn/simple
```

- 启动

```
CUDA_VISIBLE_DEVICES=0,1,2,3,4,5,6,7 vllm serve ./QwQ-32B --tensor-parallel-size 8 --port 8081
```

  1. CUDA_VISIBLE_DEVICES=0,1,2,3,4,5,6,7 :指定使用的 GPU 设备为第1-8张卡。
  2. vllm serve:启动一个 HTTP 服务器，用于接收和处理推理请求。
  3. ./QwQ-32B： 指定要加载的模型路径。
  4. --tensor-parallel-size 8： 设置张量并行的大小。8 表示使用 8 个 GPU 进行张量并行计算。
  5. --port 8081：指定HTTP服务监听的端口号。

- 调用

```
from openai import OpenAI
import openai

openai.api_key = '1111111' # 这里随便填一个
openai.base_url = 'http://127.0.0.1:8081/v1'


def get_completion(prompt, model="QwQ-32B"):
    client = OpenAI(api_key=openai.api_key,
                    base_url=openai.base_url
                    )
    messages = [{"role": "user", "content": prompt}]
    response = client.chat.completions.create(
        model=model,
        messages=messages,
        stream=False
    )
    return response.choices[0].message.content

prompt = '你好，请幽默的介绍下你自己，不少于300子'
get_completion(prompt, model="./QwQ-32B")
```

6.6 sglang安装+启动

- 使用conda创建虚拟环境

```
conda create -n sglang python=3.12

conda activate sglang
```

- 下载vllm（指定源，否则极慢）

```
pip install vllm -i https://pypi.tuna.tsinghua.edu.cn/simple
```

- 下载sglang

```
pip install sglang -i https://pypi.tuna.tsinghua.edu.cn/simple
pip install sgl_kernel -i https://pypi.tuna.tsinghua.edu.cn/simple
```

- 下载其他依赖

```
pip install orjson
pip install torchao
```

- 启动

```
python -m sglang.launch_server \
  --model-path /root/Hugging-Face/hub/models--deepseek-ai--DeepSeek-R1-Distill-Qwen-7B/snapshots/916b56a44061fd5cd7d6a8fb632557ed4f724f60 \
  --host 0.0.0.0 \
  --port 8081
```

- 调用

```
from openai import OpenAI
import openai

openai.api_key = '1111111' # 这里随便填一个
openai.base_url = 'http://127.0.0.1:8081/v1'


def get_completion(prompt, model="QwQ-32B"):
    client = OpenAI(api_key=openai.api_key,
                    base_url=openai.base_url
                    )
    messages = [{"role": "user", "content": prompt}]
    response = client.chat.completions.create(
        model=model,
        messages=messages,
        stream=False
    )
    return response.choices[0].message.content
```

```python

prompt = '你好，请简短并幽默的介绍下你自己'


response = get_completion(prompt,
                          model="DeepSeek-R1-Distill-Qwen-7B"
                    )
print(response)

```

6.7 npu-smi info

-

7 mcp

7.1 uv

- 创建环境

```
pip install uv
```

或者curl

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

- 安装python依赖

```
uv pip install requests
```

与 `pip install requests` 类似，但更快。

- 创建虚拟环境

虚拟环境名称：mcp

```
uv venv mcp
```

等效于 `python -m venv mcp`，但更高效。

- 激活虚拟环境

```
source mcp/bin/activate  # Linux/macOS

mcp\Scripts\activate     # Windows
```

- 安装requirements.txt

```
uv pip install -r requirements.txt
```

- 直接运行python项目

如果项目中包含 `pyproject.toml`，你可以直接运行：

```
uv run python script.py
```

这等效于：

```
pip install -r requirements.txt
python script.py
```

7.2 MCP极简客户端搭建流程

- 创建 MCP 客户端项目

```
# 创建项目目录
uv init mcp-client
```

- 创建mcp客户端虚拟环境

```
cd mcp-client

# 创建虚拟环境
uv venv

# 激活虚拟环境
source .venv/bin/activate # Linux/macOS
.venv\Scripts\activate    # windows
```